STM LAB – ALL EXPERIMENT CODES (1–10)

Exam-ready code collection based on the STM Lab Manual.

Based on STM Lab Manual Final (1).pdf.

Note: Experiment 1 is performed in Selenium IDE, not as a Java program.

Experiment 7(d) Excel is omitted, as previously requested. Experiment 7 includes 7(a), 7(b), 7(c), 7(e), 7(f).

EXPERIMENT 1 – RECORDING IN CONTEXT SENSITIVE MODE AND ANALOG MODE

Selenium IDE steps (no Java code):

1. Open Chrome/Edge and install Selenium IDE extension.
2. Open Selenium IDE.
3. Create a new project.
4. Add a new test case.
5. Click Record.
6. Enter base URL (example: https://www.google.com) and start recording.
7. Click the search box, type a keyword, press Enter, click a result, and go back.
8. Stop recording and review commands (open, click, type, assert).
9. Run the test using the Play button.
10. For analog-style actions, record a drag-and-drop or slider movement on a demo site.
11. Save the project.

Example Selenium IDE commands:

```
open | https://www.google.com |
type | name=q | selenium
assertTitle | Google |
```

EXPERIMENT 2 – GUI CHECKPOINT FOR SINGLE PROPERTY (TITLE)

```
import javax.swing.JFrame;

public class Exp2_SinglePropertyCheckpoint {
    public static void main(String[] args) throws InterruptedException {
        JFrame frame = new JFrame("Google");
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);

        String expectedTitle = "Google";
        String actualTitle = frame.getTitle();

        System.out.println("Actual Title: " + actualTitle);
        System.out.println("Expected Title: " + expectedTitle);

        if (actualTitle.equals(expectedTitle)) {
            System.out.println("PASS: Title is correct");
        } else {
            System.out.println("FAIL: Title is incorrect");
        }
```

```
        Thread.sleep(2000);
        frame.dispose();
    }
}
```

EXPERIMENT 3 – GUI CHECKPOINT FOR SINGLE OBJECT (DISPLAYED/ENABLED)

```
import javax.swing.*;

public class Exp3_SingleObjectCheckpoint {
    public static void main(String[] args) throws InterruptedException {
        JFrame frame = new JFrame("Single Object Checkpoint");
        JTextField searchBox = new JTextField(20);

        JPanel panel = new JPanel();
        panel.add(new JLabel("Search:"));
        panel.add(searchBox);
        frame.add(panel);

        frame.setSize(400, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        Thread.sleep(500);

        boolean displayed = searchBox.isShowing();
        boolean enabled = searchBox.isEnabled();

        System.out.println("Displayed: " + displayed);
        System.out.println("Enabled: " + enabled);

        System.out.println(displayed && enabled
                ? "PASS: Search box is ready"
                : "FAIL: Search box is not ready");

        Thread.sleep(2000);
        frame.dispose();
    }
}
```

EXPERIMENT 4(a) – BITMAP CHECKPOINT FOR OBJECT/WINDOW

```
import javax.swing.*;
import javax.imageio.ImageIO;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;

public class BitmapObjectCheckpoint {
    public static void main(String[] args) throws Exception {
        JFrame frame = new JFrame("Bitmap Checkpoint Test");
        JTextField searchBox = new JTextField("Search here", 20);
```

```
    JPanel panel = new JPanel();
    panel.add(searchBox);
    frame.add(panel);

    frame.setSize(350, 120);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
    Thread.sleep(1000);

    File folder = new File("Screenshots");
    if (!folder.exists()) folder.mkdir();

    BufferedImage currentImage = new BufferedImage(
            searchBox.getWidth(), searchBox.getHeight(),
            BufferedImage.TYPE_INT_RGB);

    Graphics2D g2 = currentImage.createGraphics();
    searchBox.paint(g2);
    g2.dispose();

    File currentFile = new File("Screenshots/current_searchbox.png");
    ImageIO.write(currentImage, "png", currentFile);
    System.out.println("Current screenshot saved: " + currentFile.getAbsolutePath());

    File baselineFile = new File("Screenshots/baseline_searchbox.png");

    if (!baselineFile.exists()) {
        System.out.println("Baseline image not found.");
        System.out.println("Rename current_searchbox.png as baseline_searchbox.png");
    } else {
        boolean result = compareImages(baselineFile, currentFile);
        if (result) {
            System.out.println("BITMAP CHECKPOINT: TEST PASS");
            System.out.println("Images are identical.");
        } else {
            System.out.println("BITMAP CHECKPOINT: TEST FAIL");
            System.out.println("Images are different.");
        }
    }

    Thread.sleep(2000);
    frame.dispose();
}

public static boolean compareImages(File img1, File img2) throws Exception {
    BufferedImage image1 = ImageIO.read(img1);
    BufferedImage image2 = ImageIO.read(img2);

    if (image1.getWidth() != image2.getWidth()
            || image1.getHeight() != image2.getHeight()) {
```

```
            return false;
        }
        for (int x = 0; x < image1.getWidth(); x++) {
            for (int y = 0; y < image1.getHeight(); y++) {
                if (image1.getRGB(x, y) != image2.getRGB(x, y)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```

EXPERIMENT 4(b) – BITMAP CHECKPOINT FOR SCREEN AREA (CROP + COMPARE)

```
import javax.swing.*;
import javax.imageio.ImageIO;
import java.awt.*;
import java.awt.image.BufferedImage;
import java.io.File;

public class BitmapAreaCheckpoint {
    public static void main(String[] args) throws Exception {
        JFrame frame = new JFrame("Bitmap Area Checkpoint");
        JPanel panel = new JPanel(new FlowLayout());

        panel.add(new JLabel("Google Search"));
        panel.add(new JTextField(25));
        panel.add(new JButton("Search"));

        frame.add(panel);
        frame.setSize(800, 600);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
        Thread.sleep(1000);

        BufferedImage fullImage = new BufferedImage(
                frame.getWidth(), frame.getHeight(),
                BufferedImage.TYPE_INT_RGB);

        Graphics2D g2 = fullImage.createGraphics();
        frame.paint(g2);
        g2.dispose();

        File folder = new File("SeleniumScreenshots");
        if (!folder.exists()) folder.mkdirs();

        int width = Math.min(800, fullImage.getWidth());
        int height = Math.min(600, fullImage.getHeight());
```

```
        BufferedImage currentArea = fullImage.getSubimage(0, 0, width, height);

        File currentFile = new File("SeleniumScreenshots/current_google_area.png");
        ImageIO.write(currentArea, "png", currentFile);
        System.out.println("Current screenshot saved: " + currentFile.getAbsolutePath());

        File baselineFile = new File("SeleniumScreenshots/baseline_google_area.png");

        if (!baselineFile.exists()) {
            System.out.println("Baseline image not found.");
            System.out.println("Rename current_google_area.png to baseline_google_area.png");
        } else {
            BufferedImage baseline = ImageIO.read(baselineFile);
            double similarity = compareImages(baseline, currentArea);
            System.out.printf("Image Similarity: %.2f%%%n", similarity);

            if (similarity >= 95)
                System.out.println("BITMAP CHECKPOINT: TEST PASS");
            else
                System.out.println("BITMAP CHECKPOINT: TEST FAIL");
        }

        Thread.sleep(2000);
        frame.dispose();
    }

    public static double compareImages(BufferedImage img1, BufferedImage img2) {
        if (img1.getWidth() != img2.getWidth()
                || img1.getHeight() != img2.getHeight()) return 0;

        long totalPixels = (long) img1.getWidth() * img1.getHeight();
        long samePixels = 0;

        for (int x = 0; x < img1.getWidth(); x++) {
            for (int y = 0; y < img1.getHeight(); y++) {
                if (img1.getRGB(x, y) == img2.getRGB(x, y)) samePixels++;
            }
        }
        return ((double) samePixels / totalPixels) * 100;
    }
}
```

EXPERIMENT 5 – DATABASE CHECKPOINT (DEFAULT CHECK / RECORD EXISTS)

```
import java.sql.*;

public class DefaultDatabaseCheckpoint {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                    "jdbc:h2:./StudentDB", "sa", "");
```

```
            Statement stmt = con.createStatement();

            stmt.execute("CREATE TABLE IF NOT EXISTS STUDENT ("
                    + "ID INT PRIMARY KEY, "
                    + "NAME VARCHAR(50), "
                    + "EMAIL VARCHAR(100))");

            stmt.execute("DELETE FROM STUDENT");

            stmt.executeUpdate("INSERT INTO STUDENT VALUES "
                    + "(1, 'Ajay', 'ajay@gmail.com')");

            ResultSet rs = stmt.executeQuery(
                    "SELECT * FROM STUDENT WHERE ID = 1");

            if (rs.next()) {
                System.out.println("Record Exists");
                System.out.println("ID: " + rs.getInt("ID"));
                System.out.println("Name: " + rs.getString("NAME"));
                System.out.println("Email: " + rs.getString("EMAIL"));
                System.out.println("TEST PASSED");
            } else {
                System.out.println("Record Not Found");
                System.out.println("TEST FAILED");
            }

            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

EXPERIMENT 6 – DATABASE CHECKPOINT (CUSTOM CHECK)

```
import java.sql.*;

public class DatabaseCheckpointCustomCheck {
    static final String DB_URL = "jdbc:sqlite:student.db";

    public static void setupDatabase() throws SQLException {
        try (Connection conn = DriverManager.getConnection(DB_URL);
             Statement stmt = conn.createStatement()) {

            stmt.execute("CREATE TABLE IF NOT EXISTS students ("
                    + "id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)");

            stmt.execute("DELETE FROM students");
            stmt.execute("INSERT INTO students VALUES (1, 'Amit', 78)");
            stmt.execute("INSERT INTO students VALUES (2, 'Neha', 45)");
```

```
            stmt.execute("INSERT INTO students VALUES (3, 'Ravi', 92)");
        }
    }

    public static boolean customCheckpoint(int studentId, String expectedStatus)
            throws SQLException {

        try (Connection conn = DriverManager.getConnection(DB_URL);
             PreparedStatement ps = conn.prepareStatement(
                     "SELECT marks FROM students WHERE id = ?")) {

            ps.setInt(1, studentId);
            ResultSet rs = ps.executeQuery();

            if (!rs.next()) {
                System.out.println("FAIL: No record found for id " + studentId);
                return false;
            }

            int marks = rs.getInt("marks");
            String actualStatus = (marks >= 50) ? "Pass" : "Fail";

            if (actualStatus.equals(expectedStatus)) {
                System.out.println("PASS: Student " + studentId
                        + " status = " + actualStatus + " (Custom check matched)");
                return true;
            } else {
                System.out.println("FAIL: Student " + studentId
                        + " expected " + expectedStatus + ", got " + actualStatus);
                return false;
            }
        }
    }

    public static void main(String[] args) throws Exception {
        setupDatabase();
        customCheckpoint(1, "Pass");
        customCheckpoint(2, "Fail");
        customCheckpoint(3, "Pass");
    }
}
```

EXPERIMENT 7(a) – DATA DRIVEN TEST (DYNAMIC DATA)

```
import java.util.Random;

public class DynamicDataTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
```

```
            default: return price;
        }
    }

    public static void main(String[] args) {
        String[] categories = {"electronics", "clothing", "grocery"};
        Random rand = new Random();

        System.out.println("---- Dynamic Test Data Submission ----");

        for (int i = 1; i <= 5; i++) {
            int price = 100 + rand.nextInt(901);
            String category = categories[rand.nextInt(categories.length)];
            double result = calculateDiscount(price, category);

            System.out.printf("Test %d: price=%d, category=%s -> discounted=%.2f%n",
                    i, price, category, result);
        }
    }
}
```

EXPERIMENT 7(b) – DATA DRIVEN TEST THROUGH FLAT FILE

```
import java.io.*;

public class FlatFileTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createFlatFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("testdata.txt"))) {
            pw.println("500,electronics");
            pw.println("300,clothing");
            pw.println("150,grocery");
        }
    }

    public static void flatFileTest() throws IOException {
        System.out.println("---- Data Driven Test via Flat File ----");

        try (BufferedReader br = new BufferedReader(new FileReader("testdata.txt"))) {
            String line;

            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                double price = Double.parseDouble(parts[0]);
                String category = parts[1];
```

```
                double result = calculateDiscount(price, category);

                System.out.printf("price=%.1f, category=%s -> discounted=%.2f%n",
                        price, category, result);
            }
        }
    }

    public static void main(String[] args) throws IOException {
        createFlatFile();
        flatFileTest();
    }
}
```

EXPERIMENT 7(c) – DATA DRIVEN TEST THROUGH FRONT GRID

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class FrontEndGridTest {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void main(String[] args) {
        JFrame frame = new JFrame("Front End Grid - Data Driven Test");

        String[] columns = {"Price", "Category", "Result"};
        DefaultTableModel model = new DefaultTableModel(columns, 0);

        Object[][] gridData = {
                {500.0, "electronics"},
                {300.0, "clothing"},
                {150.0, "grocery"}
        };

        for (Object[] row : gridData) {
            double price = (double) row[0];
            String category = (String) row[1];
            double result = calculateDiscount(price, category);

            model.addRow(new Object[]{
                    price, category, String.format("%.2f", result)
            });
        }

        JTable table = new JTable(model);
        frame.add(new JScrollPane(table));
```

```
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

EXPERIMENT 7(e) – BATCH TESTING WITHOUT PARAMETERS

```
public class BatchNoParams {
    public static void testAddition() {
        assert 2 + 3 == 5;
        System.out.println("test_addition: PASS");
    }

    public static void testSubtraction() {
        assert 5 - 2 == 3;
        System.out.println("test_subtraction: PASS");
    }

    public static void main(String[] args) {
        System.out.println("---- Batch Testing (No Parameters) ----");
        testAddition();
        testSubtraction();
    }
}
```

```
Run with assertions enabled:
javac BatchNoParams.java
java -ea BatchNoParams
```

EXPERIMENT 7(f) – BATCH TESTING WITH PARAMETERS

```
public class BatchWithParams {
    static void testOperation(int a, int b, char op, int expected) {
        int result = 0;

        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
        }

        String status = (result == expected) ? "PASS" : "FAIL";

        System.out.printf("Test: %d %c %d = %d (expected %d) -> %s%n",
                a, op, b, result, expected, status);
    }

    public static void main(String[] args) {
        int[][] testCases = {
                {2, 3, '+', 5},
```

```
                {10, 4, '-', 6},
                {3, 3, '*', 9},
                {5, 5, '-', 1}
        };

        System.out.println("---- Batch Testing (With Parameters) ----");

        for (int[] tc : testCases) {
            testOperation(tc[0], tc[1], (char) tc[2], tc[3]);
        }
    }
}
```

EXPERIMENT 8 – DATA DRIVEN BATCH

```
import java.io.*;

public class DataDrivenBatch {
    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createCsvFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("batch_data.csv"))) {
            pw.println("Price,Category,ExpectedResult");
            pw.println("500,electronics,450");
            pw.println("300,clothing,240");
            pw.println("150,grocery,150");
            pw.println("1000,electronics,900");
        }
    }

    public static void dataDrivenBatch() throws IOException {
        System.out.println("---- Data Driven Batch Test ----");

        int passCount = 0, failCount = 0;

        try (BufferedReader br = new BufferedReader(new FileReader("batch_data.csv"))) {
            String line = br.readLine(); // Skip header

            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                double price = Double.parseDouble(parts[0]);
                String category = parts[1];
                double expected = Double.parseDouble(parts[2]);

                double actual = calculateDiscount(price, category);
```

```
                if (actual == expected) {
                    System.out.printf("PASS: %s, price=%.1f -> %.1f%n",
                            category, price, actual);
                    passCount++;
                } else {
                    System.out.printf("FAIL: %s, price=%.1f -> got %.1f, expected %.1f%n",
                            category, price, actual, expected);
                    failCount++;
                }
            }
        }

        System.out.println("Batch Summary: " + passCount
                + " Passed, " + failCount + " Failed");
    }

    public static void main(String[] args) throws IOException {
        createCsvFile();
        dataDrivenBatch();
    }
}
```

EXPERIMENT 9 – SILENT MODE TEST EXECUTION

```
import java.util.logging.*;
import java.io.IOException;

public class SilentModeTest {
    static Logger logger = Logger.getLogger("SilentTestLogger");

    static void setupLogger() throws IOException {
        logger.setUseParentHandlers(false);

        FileHandler fh = new FileHandler("silent_test_log.txt");
        fh.setFormatter(new SimpleFormatter());
        logger.addHandler(fh);
    }

    static void testCase(String name, Object actual, Object expected) {
        try {
            if (actual.equals(expected)) {
                logger.info(name + ": PASS (actual=" + actual
                        + ", expected=" + expected + ")");
            } else {
                logger.severe(name + ": FAIL (actual=" + actual
                        + ", expected=" + expected + ")");
            }
        } catch (Exception e) {
            logger.severe(name + ": ERROR - " + e.getMessage());
        }
```

```
    }

    public static void main(String[] args) throws IOException {
        setupLogger();

        testCase("Addition Test", 2 + 2, 4);
        testCase("Subtraction Test", 10 - 5, 5);
        testCase("Multiplication Test", 3 * 3, 10);
        testCase("String Test", "abc".toUpperCase(), "ABC");

        System.out.println("Batch execution completed silently. "
                + "Check silent_test_log.txt for results.");
    }
}
```

EXPERIMENT 10 – CALCULATOR TEST IN WINDOWS APPLICATION

```
import java.awt.*;
import java.awt.event.KeyEvent;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;

public class CalculatorTest {
    public static void main(String[] args) throws Exception {
        Runtime.getRuntime().exec("calc.exe");
        Thread.sleep(2000);

        Robot robot = new Robot();
        robot.setAutoDelay(200);

        typeKey(robot, KeyEvent.VK_7);
        typeKey(robot, KeyEvent.VK_ADD);
        typeKey(robot, KeyEvent.VK_5);
        typeKey(robot, KeyEvent.VK_EQUALS);

        Thread.sleep(1000);

        Rectangle screenRect = new Rectangle(
                Toolkit.getDefaultToolkit().getScreenSize());

        BufferedImage capture = robot.createScreenCapture(screenRect);

        ImageIO.write(capture, "png", new File("calculator_result.png"));

        System.out.println("Test executed: 7 + 5 =");
        System.out.println("Screenshot saved as calculator_result.png -- "
                + "compare visually to confirm result is 12");
    }

    static void typeKey(Robot robot, int keyCode) {
        robot.keyPress(keyCode);
        robot.keyRelease(keyCode);
    }
```

```
}
```

IMPORTANT SETUP NOTES

1. Experiments 2, 3, 4(a), 4(b), 7(c) use Swing GUI.
2. Experiment 5 needs the H2 JDBC driver/library in the project.
3. Experiment 6 needs the SQLite JDBC driver/library in the project.
4. Experiment 7(e): run with -ea to enable Java assertions.
5. Experiment 10 is for Windows because it launches calc.exe.
6. For 4(a), create/save a baseline image first as Screenshots/baseline_searchbox.png.
7. For 4(b), create/save a baseline image first as SeleniumScreenshots/baseline_google_area.png.
8. Each public class should be saved in a .java file with the same class name.